

Excelente! Com a arquitetura bem definida e as principais classes utilitárias e de modelo esboçadas, o próximo passo lógico é a implementação do StatsDAO para carregar os dados existentes do MegaPower.db. Este DAO será fundamental para que as estratégias estatísticas possam funcionar.

Implementação do StatsDAO e Modelos Auxiliares

Primeiro, vamos definir as classes de modelo para as tabelas existentes que o StatsDAO irá mapear: Sorteio, EstatisticaGeralDezena e EstMudEstado. Em seguida, implementaremos o StatsDAO para carregar esses dados e fornecer métodos para que as estratégias possam acessá-los de forma eficiente.

1. Modelos para Tabelas Existentes

- Sorteio.java: Representa um registro da tabela SORTEIOS.

```
Java
package com.megasena.model;

import java.time.LocalDate;
import java.util.Arrays;
import java.util.List;
import java.util.Collections;
import java.util.Set;
import java.util.TreeSet;
import java.util.stream.Collectors;

public class Sorteio {
    private int idSorteio;
    private int concurso;
    private LocalDate dataSorteio;
    private Set<Integer> dezenasSorteadas; // Usar Set para garantir unicidade e ordenação
    private double premioSena;
    private double premioQuina;
    private double premioQuadra;
    private int ganhadoresSena;
    private int ganhadoresQuina;
    private int ganhadoresQuadra;
    private boolean acumulado;
    private double valorAcumulado;

    public Sorteio(int idSorteio, int concurso, LocalDate dataSorteio, List<Integer>
dezenasSorteadas,
        double premioSena, double premioQuina, double premioQuadra,
        int ganhadoresSena, int ganhadoresQuina, int ganhadoresQuadra,
```

```

        boolean acumulado, double valorAcumulado) {
    this.idSorteio = idSorteio;
    this.concurso = concurso;
    this.dataSorteio = dataSorteio;
    // Garantir que as dezenas estejam ordenadas e sejam únicas
    this.dezenasSorteadas = new TreeSet<>(dezenasSorteadas);
    this.premioSena = premioSena;
    this.premioQuina = premioQuina;
    this.premioQuadra = premioQuadra;
    this.ganhadoresSena = ganhadoresSena;
    this.ganhadoresQuina = ganhadoresQuina;
    this.ganhadoresQuadra = ganhadoresQuadra;
    this.acumulado = acumulado;
    this.valorAcumulado = valorAcumulado;
}

// Getters
public int getIdSorteio() { return idSorteio; }
public int getConcurso() { return concurso; }
public LocalDate getDataSorteio() { return dataSorteio; }
public Set<Integer> getDezenasSorteadas() { return
Collections.unmodifiableSet(dezenasSorteadas); } // Retorna uma visão imutável
public double getPremioSena() { return premioSena; }
public double getPremioQuina() { return premioQuina; }
public double getPremioQuadra() { return premioQuadra; }
public int getGanhadoresSena() { return ganhadoresSena; }
public int getGanhadoresQuina() { return ganhadoresQuina; }
public int getGanhadoresQuadra() { return ganhadoresQuadra; }
public boolean isAcumulado() { return acumulado; }
public double getValorAcumulado() { return valorAcumulado; }

@Override
public String toString() {
    return "Sorteio{" +
        "concurso=" + concurso +
        ", dataSorteio=" + dataSorteio +
        ", dezenas=" +
dezenasSorteadas.stream().map(Object::toString).collect(Collectors.joining(", ")) +
        '}';
}
}

```

- EstatisticaGeralDezena.java: Representa um registro da view/tabela

ESTATISTICAS_GERAIS_DEZENAS.

Java

```
package com.megasena.model;
```

```
public class EstatisticaGeralDezena {
```

```
    private int dezena;
```

```
    private int quantidadeOcorrencias;
```

```
    private int ultimoSorteio;
```

```
    private int concursosAusente;
```

```
    public EstatisticaGeralDezena(int dezena, int quantidadeOcorrencias, int ultimoSorteio, int  
concursosAusente) {
```

```
        this.dezena = dezena;
```

```
        this.quantidadeOcorrencias = quantidadeOcorrencias;
```

```
        this.ultimoSorteio = ultimoSorteio;
```

```
        this.concursosAusente = concursosAusente;
```

```
    }
```

```
    // Getters
```

```
    public int getDezena() { return dezena; }
```

```
    public int getQuantidadeOcorrencias() { return quantidadeOcorrencias; }
```

```
    public int getUltimoSorteio() { return ultimoSorteio; }
```

```
    public int getConcursosAusente() { return concursosAusente; }
```

```
@Override
```

```
public String toString() {
```

```
    return "EstatisticaGeralDezena{" +
```

```
        "dezena=" + dezena +
```

```
        ", ocorrencias=" + quantidadeOcorrencias +
```

```
        ", ultimoSorteio=" + ultimoSorteio +
```

```
        ", concursosAusente=" + concursosAusente +
```

```
        '}';
```

```
    }
```

```
}
```

- EstMudEstado.java: Representa um registro da tabela EST_MUD_ESTADO.

Java

```
package com.megasena.model;
```

```
public class EstMudEstado {
```

```
    private int concursoAnterior;
```

```
    private int dezenaAnterior;
```

```
    private int concursoAtual;
```

```

private int dezenaAtual;
private int transicaoCount;

public EstMudEstado(int concursoAnterior, int dezenaAnterior, int concursoAtual, int
dezenaAtual, int transicaoCount) {
    this.concursoAnterior = concursoAnterior;
    this.dezenaAnterior = dezenaAnterior;
    this.concursoAtual = concursoAtual;
    this.dezenaAtual = dezenaAtual;
    this.transicaoCount = transicaoCount;
}

// Getters
public int getConcursoAnterior() { return concursoAnterior; }
public int getDezenaAnterior() { return dezenaAnterior; }
public int getConcursoAtual() { return concursoAtual; }
public int getDezenaAtual() { return dezenaAtual; }
public int getTransicaoCount() { return transicaoCount; }

@Override
public String toString() {
    return "EstMudEstado{" +
        "dezenaAnterior=" + dezenaAnterior +
        " -> dezenaAtual=" + dezenaAtual +
        ", count=" + transicaoCount +
        '}';
}
}

```

2. Implementação do StatsDAO

O StatsDAO será responsável por buscar os dados estatísticos e históricos do MegaPower.db.

Java

```

package com.megasena.dao;

import com.megasena.model.Sorteio;
import com.megasena.model.EstatisticaGeralDezena;

```

```

import com.megasena.model.EstMudEstado;
import com.megasena.util.DatabaseManager;
import com.megasena.util.RetryMechanism;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class StatsDAO {
    private final DatabaseManager dbManager;

    public StatsDAO(DatabaseManager dbManager) {
        this.dbManager = dbManager;
    }

    /**
     * Retorna a frequência de todas as dezenas.
     * Mapeia dezena -> quantidade de ocorrências.
     * @return Map<Integer, Integer> com a frequência de cada dezena.
     * @throws SQLException Se ocorrer um erro de banco de dados.
     */
    public Map<Integer, Integer> getFrequenciaTodasDezenas() throws SQLException {
        Map<Integer, Integer> frequencia = new HashMap<>();
        String sql = "SELECT DEZENA, QUANTIDADE_OCORRENCIAS FROM ESTATISTICAS_GERAIS_DEZENAS ORDER BY DEZENA";
        try (Connection conn = dbManager.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql);
            ResultSet rs = RetryMechanism.executeWithRetry(() -> pstmt.executeQuery())) { // Usar
retry
            while (rs.next()) {
                frequencia.put(rs.getInt("DEZENA"), rs.getInt("QUANTIDADE_OCORRENCIAS"));
            }
        }
        return frequencia;
    }
}

```

```

/**
 * Retorna o atraso atual de todas as dezenas.
 * Mapeia dezena -> concursos ausente.
 * @return Map<Integer, Integer> com o atraso de cada dezena.
 * @throws SQLException Se ocorrer um erro de banco de dados.
 */
public Map<Integer, Integer> getAtrasoTodasDezenas() throws SQLException {
    Map<Integer, Integer> atraso = new HashMap<>();
    String sql = "SELECT DEZENA, CONCURSOS_AUSENTE FROM ESTATISTICAS_GERAIS_DEZENAS
ORDER BY DEZENA";
    try (Connection conn = dbManager.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql);
        ResultSet rs = RetryMechanism.executeWithRetry(() -> pstmt.executeQuery())) {
        while (rs.next()) {
            atraso.put(rs.getInt("DEZENA"), rs.getInt("CONCURSOS_AUSENTE"));
        }
    }
    return atraso;
}

/**
 * Retorna a frequência de dezenas por posição.
 * Mapeia dezena -> posição -> quantidade.
 * @return Map<Integer, Map<Integer, Integer>> com a frequência de dezenas por posição.
 * @throws SQLException Se ocorrer um erro de banco de dados.
 */
public Map<Integer, Map<Integer, Integer>> getFrequenciaDezenasPorPosicao() throws
SQLException {
    Map<Integer, Map<Integer, Integer>> frequenciaPorPosicao = new HashMap<>();
    String sql = "SELECT DEZENA, POSICAO, QUANTIDADE FROM AGREG_DIS_DEZ_POSICAO ORDER
BY DEZENA, POSICAO";
    try (Connection conn = dbManager.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql);
        ResultSet rs = RetryMechanism.executeWithRetry(() -> pstmt.executeQuery())) {
        while (rs.next()) {
            int dezena = rs.getInt("DEZENA");
            int posicao = rs.getInt("POSICAO");
            int quantidade = rs.getInt("QUANTIDADE");
            frequenciaPorPosicao.computeIfAbsent(dezena, k -> new HashMap<>()).put(posicao,
quantidade);
        }
    }
    return frequenciaPorPosicao;
}

```

```

/**
 * Retorna todos os sorteios históricos.
 * @return List<Sorteio> lista de todos os sorteios.
 * @throws SQLException Se ocorrer um erro de banco de dados.
 */
public List<Sorteio> getAllSorteios() throws SQLException {
    List<Sorteio> sorteios = new ArrayList<>();
    String sql = "SELECT ID_SORTEIO, CONCURSO, DATA_SORTEIO, DEZENA_1, DEZENA_2,
DEZENA_3, DEZENA_4, DEZENA_5, DEZENA_6, " +
        "PREMIO_SENA, PREMIO_QUINA, PREMIO_QUADRA, GANHADORES_SENA,
GANHADORES_QUINA, GANHADORES_QUADRA, " +
        "ACUMULADO, VALOR_ACUMULADO FROM SORTEIOS ORDER BY CONCURSO ASC";
    try (Connection conn = dbManager.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql);
        ResultSet rs = RetryMechanism.executeWithRetry(() -> pstmt.executeQuery())) {
        while (rs.next()) {
            List<Integer> dezenas = Arrays.asList(
                rs.getInt("DEZENA_1"), rs.getInt("DEZENA_2"), rs.getInt("DEZENA_3"),
                rs.getInt("DEZENA_4"), rs.getInt("DEZENA_5"), rs.getInt("DEZENA_6")
            );
            sorteios.add(new Sorteio(
                rs.getInt("ID_SORTEIO"),
                rs.getInt("CONCURSO"),
                LocalDate.parse(rs.getString("DATA_SORTEIO")), // Assumindo formato YYYY-MM-DD
                dezenas,
                rs.getDouble("PREMIO_SENA"),
                rs.getDouble("PREMIO_QUINA"),
                rs.getDouble("PREMIO_QUADRA"),
                rs.getInt("GANHADORES_SENA"),
                rs.getInt("GANHADORES_QUINA"),
                rs.getInt("GANHADORES_QUADRA"),
                "S".equalsIgnoreCase(rs.getString("ACUMULADO")),
                rs.getDouble("VALOR_ACUMULADO")
            ));
        }
    }
    return sorteios;
}

/**
 * Retorna o último sorteio registrado no banco de dados.
 * @return O objeto Sorteio do último concurso, ou null se não houver sorteios.

```

```

    * @throws SQLException Se ocorrer um erro de banco de dados.
    */
    public Sorteio getUltimoSorteio() throws SQLException {
        Sorteio ultimoSorteio = null;
        String sql = "SELECT ID_SORTEIO, CONCURSO, DATA_SORTEIO, DEZENA_1, DEZENA_2,
DEZENA_3, DEZENA_4, DEZENA_5, DEZENA_6, " +
            "PREMIO_SENA, PREMIO_QUINA, PREMIO_QUADRA, GANHADORES_SENA,
GANHADORES_QUINA, GANHADORES_QUADRA, " +
            "ACUMULADO, VALOR_ACUMULADO FROM SORTEIOS ORDER BY CONCURSO DESC LIMIT
1";
        try (Connection conn = dbManager.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql);
            ResultSet rs = RetryMechanism.executeWithRetry(() -> pstmt.executeQuery())) {
            if (rs.next()) {
                List<Integer> dezenas = Arrays.asList(
                    rs.getInt("DEZENA_1"), rs.getInt("DEZENA_2"), rs.getInt("DEZENA_3"),
                    rs.getInt("DEZENA_4"), rs.getInt("DEZENA_5"), rs.getInt("DEZENA_6")
                );
                ultimoSorteio = new Sorteio(
                    rs.getInt("ID_SORTEIO"),
                    rs.getInt("CONCURSO"),
                    LocalDate.parse(rs.getString("DATA_SORTEIO")),
                    dezenas,
                    rs.getDouble("PREMIO_SENA"),
                    rs.getDouble("PREMIO_QUINA"),
                    rs.getDouble("PREMIO_QUADRA"),
                    rs.getInt("GANHADORES_SENA"),
                    rs.getInt("GANHADORES_QUINA"),
                    rs.getInt("GANHADORES_QUADRA"),
                    "S".equalsIgnoreCase(rs.getString("ACUMULADO")),
                    rs.getDouble("VALOR_ACUMULADO")
                );
            }
        }
        return ultimoSorteio;
    }
}

```

```
/**
```

```
 * Retorna a transição de estado das dezenas.
```

```
 * Mapeia dezenaAnterior -> Map<dezenaAtual, quantidade>.
```

```
 * @return Map<Integer, Map<Integer, Integer>> das transições.
```

```
 * @throws SQLException Se ocorrer um erro de banco de dados.
```



```

*/
public Map<Integer, Map<Integer, Integer>> getTransicoesDezena() throws SQLException {
    Map<Integer, Map<Integer, Integer>> transicoes = new HashMap<>();
    String sql = "SELECT DEZENA_ANTERIOR, DEZENA_ATUAL, TRANSICAO_COUNT FROM
EST_MUD_ESTADO ORDER BY DEZENA_ANTERIOR, DEZENA_ATUAL";
    try (Connection conn = dbManager.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql);
        ResultSet rs = RetryMechanism.executeWithRetry(() -> pstmt.executeQuery())) {
        while (rs.next()) {
            int dezenaAnterior = rs.getInt("DEZENA_ANTERIOR");
            int dezenaAtual = rs.getInt("DEZENA_ATUAL");
            int count = rs.getInt("TRANSICAO_COUNT");
            transicoes.computeIfAbsent(dezenaAnterior, k -> new
HashMap<>()).put(dezenaAtual, count);
        }
    }
    return transicoes;
}

/**
 * Retorna todas as ocorrências de dezenas por sorteio.
 * @return List<Map<String, Object>>
 * @throws SQLException
 */
public List<Map<String, Object>> getAllDezOcorrencias() throws SQLException {
    List<Map<String, Object>> ocorrencias = new ArrayList<>();
    String sql = "SELECT ID_SORTEIO, DEZENA, POSICAO, DATA_SORTEIO FROM DEZ_OCORRENCIAS
ORDER BY ID_SORTEIO, POSICAO";
    try (Connection conn = dbManager.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql);
        ResultSet rs = RetryMechanism.executeWithRetry(() -> pstmt.executeQuery())) {
        while (rs.next()) {
            Map<String, Object> ocorrencia = new HashMap<>();
            ocorrencia.put("ID_SORTEIO", rs.getInt("ID_SORTEIO"));
            ocorrencia.put("DEZENA", rs.getInt("DEZENA"));
            ocorrencia.put("POSICAO", rs.getInt("POSICAO"));
            ocorrencia.put("DATA_SORTEIO", rs.getString("DATA_SORTEIO"));
            ocorrencias.add(ocorrencia);
        }
    }
    return ocorrencias;
}
}

```

3. Classes de Modelo Auxiliares para Persistência

Precisamos de mais algumas classes de modelo para os dados que serão persistidos pelo ApostaDAO (e que foram mencionados na especificação do DB):

- ResultadoEstrategia.java:

```
Java
package com.megasena.model;

import java.time.LocalDateTime;

public class ResultadoEstrategia {
    private long idResultado; // Usado para persistência, não para lógica de negócio direta
    private long idAposta; // ID da aposta associada
    private String nomeEstrategia;
    private double pontuacao;
    private LocalDateTime dataAvaliacao;

    public ResultadoEstrategia(long idAposta, String nomeEstrategia, double pontuacao) {
        this.idAposta = idAposta;
        this.nomeEstrategia = nomeEstrategia;
        this.pontuacao = pontuacao;
        this.dataAvaliacao = LocalDateTime.now();
    }

    public ResultadoEstrategia(long idResultado, long idAposta, String nomeEstrategia, double
pontuacao, LocalDateTime dataAvaliacao) {
        this.idResultado = idResultado;
        this.idAposta = idAposta;
        this.nomeEstrategia = nomeEstrategia;
        this.pontuacao = pontuacao;
        this.dataAvaliacao = dataAvaliacao;
    }

    // Getters
    public long getIdResultado() { return idResultado; }
    public long getIdAposta() { return idAposta; }
    public String getNomeEstrategia() { return nomeEstrategia; }
    public double getPontuacao() { return pontuacao; }
    public LocalDateTime getDataAvaliacao() { return dataAvaliacao; }

    // Setters (se necessário para ORM ou reconstrução)
    public void setIdResultado(long idResultado) { this.idResultado = idResultado; }
```

```

    public void setIdAposta(long idAposta) { this.idAposta = idAposta; }
    public void setNomeEstrategia(String nomeEstrategia) { this.nomeEstrategia =
nomeEstrategia; }
    public void setPontuacao(double pontuacao) { this.pontuacao = pontuacao; }
    public void setDataAvaliacao(LocalDateTime dataAvaliacao) { this.dataAvaliacao =
dataAvaliacao; }

```

```

@Override
public String toString() {
    return "ResultadoEstrategia{" +
        "nomeEstrategia=" + nomeEstrategia + "\" +
        ", pontuacao=" + pontuacao +
        "}";
}
}

```

- RankingAposta.java:

Java

```
package com.megasena.model;
```

```
import java.time.LocalDateTime;
```

```

public class RankingAposta {
    private long idRanking;
    private long idAposta;
    private double somaPontuacaoTotal;
    private int rankingFinal;
    private LocalDateTime dataRanking;

```

```

    public RankingAposta(long idAposta, double somaPontuacaoTotal, int rankingFinal) {
        this.idAposta = idAposta;
        this.somaPontuacaoTotal = somaPontuacaoTotal;
        this.rankingFinal = rankingFinal;
        this.dataRanking = LocalDateTime.now();
    }

```

```

    public RankingAposta(long idRanking, long idAposta, double somaPontuacaoTotal, int
rankingFinal, LocalDateTime dataRanking) {
        this.idRanking = idRanking;
        this.idAposta = idAposta;
        this.somaPontuacaoTotal = somaPontuacaoTotal;
        this.rankingFinal = rankingFinal;
        this.dataRanking = dataRanking;
    }

```

```

    }

    // Getters
    public long getIdRanking() { return idRanking; }
    public long getIdAposta() { return idAposta; }
    public double getSomaPontuacaoTotal() { return somaPontuacaoTotal; }
    public int getRankingFinal() { return rankingFinal; }
    public LocalDateTime getDataRanking() { return dataRanking; }

    // Setters
    public void setIdRanking(long idRanking) { this.idRanking = idRanking; }
    public void setIdAposta(long idAposta) { this.idAposta = idAposta; }
    public void setSomaPontuacaoTotal(double somaPontuacaoTotal) { this.somaPontuacaoTotal
= somaPontuacaoTotal; }
    public void setRankingFinal(int rankingFinal) { this.rankingFinal = rankingFinal; }
    public void setDataRanking(LocalDateTime dataRanking) { this.dataRanking = dataRanking; }

    @Override
    public String toString() {
        return "RankingAposta{" +
            "idAposta=" + idAposta +
            ", pontuacaoTotal=" + somaPontuacaoTotal +
            ", rankingFinal=" + rankingFinal +
            '}';
    }
}

```

- FatAposta.java:

```

Java
package com.megasena.model;

import java.time.LocalDate;
import java.util.Collections;
import java.util.List;
import java.util.Set;
import java.util.TreeSet;

public class FatAposta {
    private long idFatAposta;
    private long idAposta;
    private LocalDate dataAposta;
    private LocalDate dataSorteio;
    private int concurso;
}

```

```

private double somaRankingEstrategias;
private Set<Integer> resultadoRealDezenas;
private double pontuacaoReal;
private int rankingResultadoReal;

public FatAposta(long idAposta, LocalDate dataAposta, LocalDate dataSorteio, int concurso,
double somaRankingEstrategias, List<Integer> resultadoRealDezenas,
double pontuacaoReal, int rankingResultadoReal) {
this.idAposta = idAposta;
this.dataAposta = dataAposta;
this.dataSorteio = dataSorteio;
this.concurso = concurso;
this.somaRankingEstrategias = somaRankingEstrategias;
this.resultadoRealDezenas = new TreeSet<>(resultadoRealDezenas);
this.pontuacaoReal = pontuacaoReal;
this.rankingResultadoReal = rankingResultadoReal;
}

// Getters
public long getIdFatAposta() { return idFatAposta; }
public long getIdAposta() { return idAposta; }
public LocalDate getDataAposta() { return dataAposta; }
public LocalDate getDataSorteio() { return dataSorteio; }
public int getConcurso() { return concurso; }
public double getSomaRankingEstrategias() { return somaRankingEstrategias; }
public Set<Integer> getResultadoRealDezenas() { return
Collections.unmodifiableSet(resultadoRealDezenas); }
public double getPontuacaoReal() { return pontuacaoReal; }
public int getRankingResultadoReal() { return rankingResultadoReal; }

// Setters (para ORM ou reconstrução)
public void setIdFatAposta(long idFatAposta) { this.idFatAposta = idFatAposta; }
public void setIdAposta(long idAposta) { this.idAposta = idAposta; }
public void setDataAposta(LocalDate dataAposta) { this.dataAposta = dataAposta; }
public void setDataSorteio(LocalDate dataSorteio) { this.dataSorteio = dataSorteio; }
public void setConcurso(int concurso) { this.concurso = concurso; }
public void setSomaRankingEstrategias(double somaRankingEstrategias) {
this.somaRankingEstrategias = somaRankingEstrategias; }
public void setResultadoRealDezenas(Set<Integer> resultadoRealDezenas) {
this.resultadoRealDezenas = new TreeSet<>(resultadoRealDezenas); }
public void setPontuacaoReal(double pontuacaoReal) { this.pontuacaoReal =
pontuacaoReal; }
public void setRankingResultadoReal(int rankingResultadoReal) { this.rankingResultadoReal =

```

```
rankingResultadoReal; }
```

```
@Override  
public String toString() {  
    return "FatAposta{" +  
        "idAposta=" + idAposta +  
        ", concurso=" + concurso +  
        ", pontuacaoReal=" + pontuacaoReal +  
        '}';  
}  
}
```

Próximos Passos

Com os DAOs e modelos para as interações com o banco de dados definidos, o próximo foco será:

1. **Implementação Completa das Estratégias:** Começaremos com as estratégias básicas (AtrasoDezenasStrategy, ParesImparesStrategy, SomaDezenasStrategy, etc.) e as integraremos com o StatsDAO.
2. **Ajustes no ApostaGenerator:** Finalizar a lógica para os modos LEXICO, FULL e FILE. O modo FULL será um gerador de combinações em *streaming* para evitar estouro de memória, enviando as apostas diretamente para avaliação.
3. **Desenvolvimento do ApostaEvaluator e RankingService:** A lógica central de pontuação e ranqueamento das apostas.
4. **Implementação do MegaSenaFacade:** Orquestrar o fluxo de geração, avaliação, ranqueamento e persistência das apostas.

Continuarei com a implementação das estratégias.